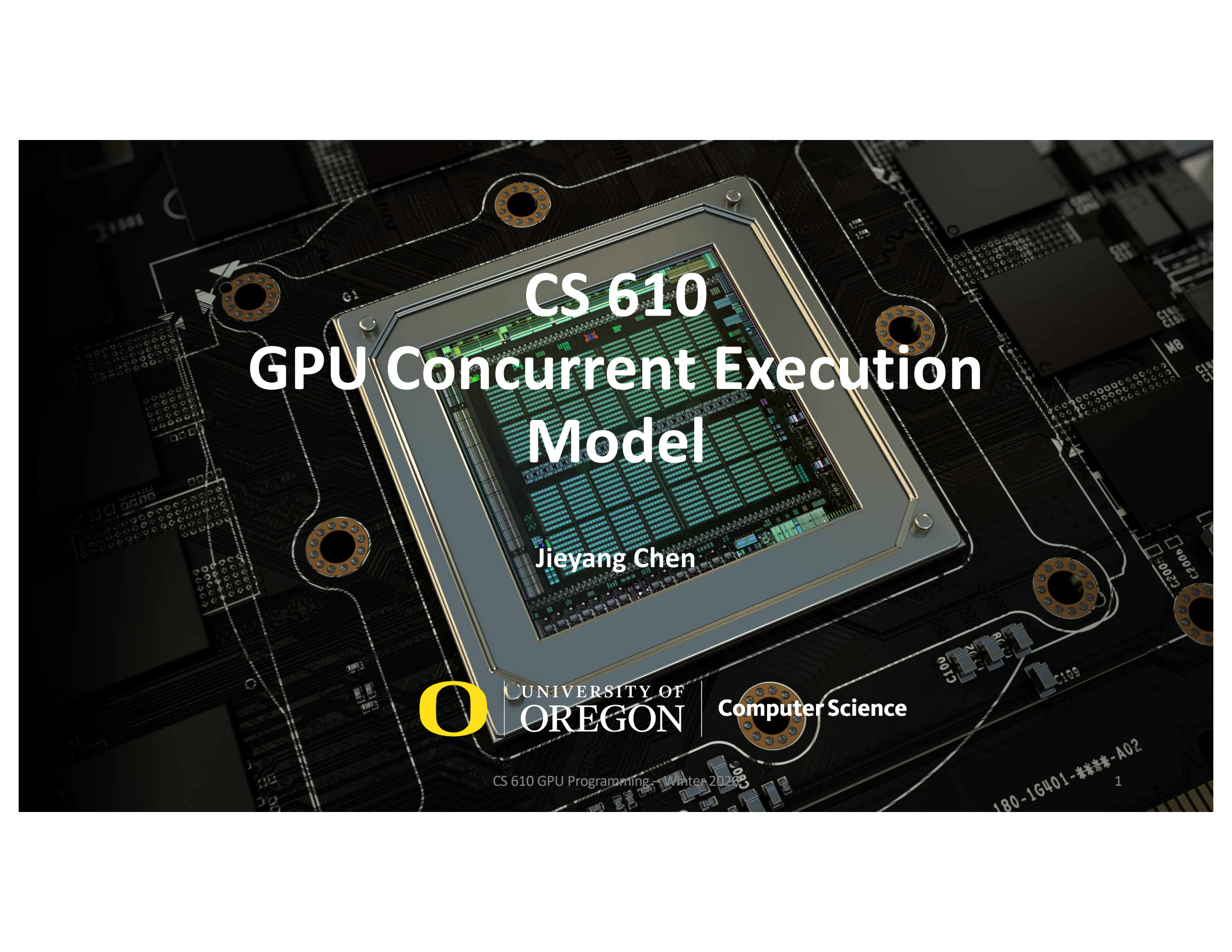




CS 610 GPU Concurrent Execution Model

Jieyang Chen



UNIVERSITY OF
OREGON

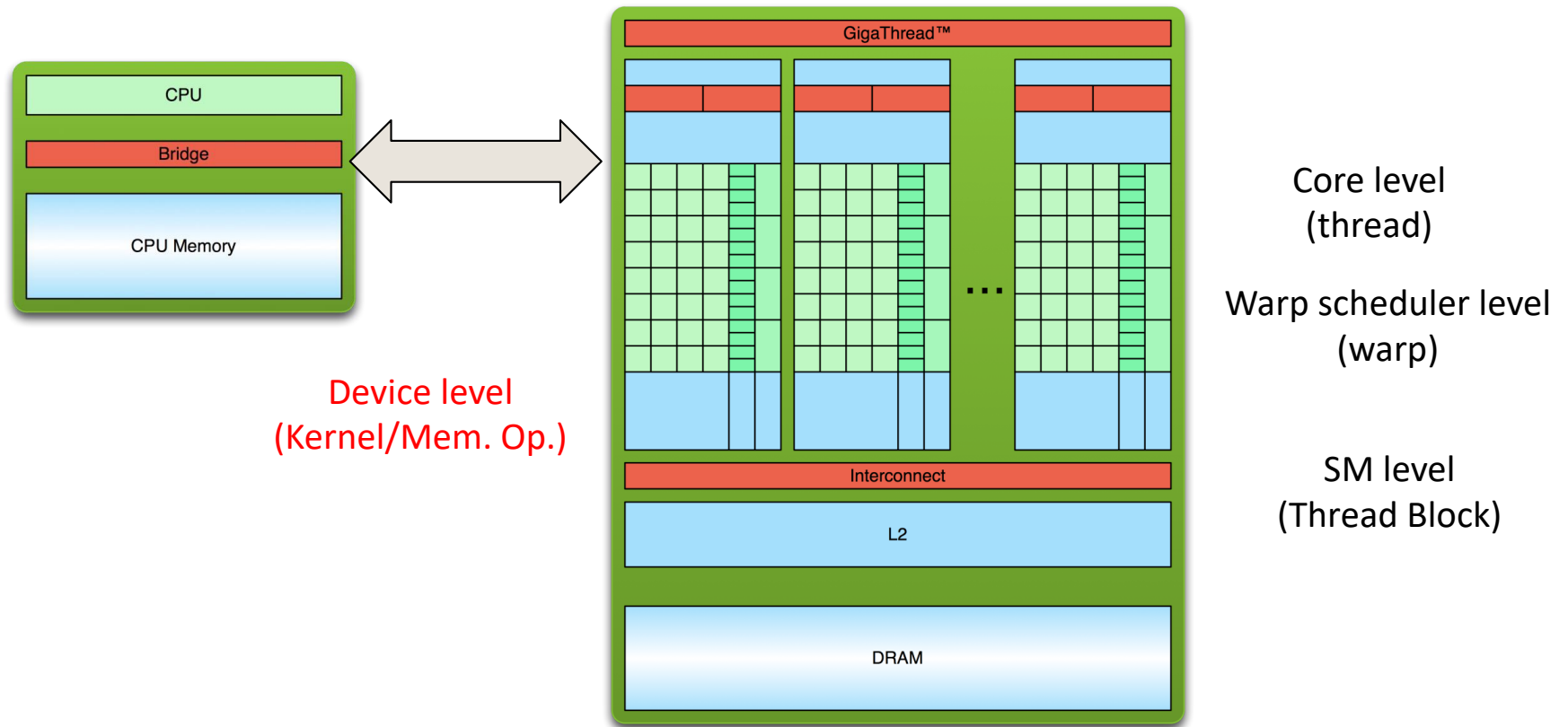
Computer Science

CS 610 GPU Programming, Winter 2026

Updates

- Coding assignment 2 due on Wednesday
- Coding option finalized
- Final project presentation schedule posted
- Submit your project proposal by Wednesday

Parallelisms



Concurrent Execution

- The ability to perform multiple CUDA operations **concurrently**
- Kernel-Kernel
- Kernel-Copy
- Kernel-CPU

Concurrency Example

- **Serial**



- **Concurrent – overlap kernels**

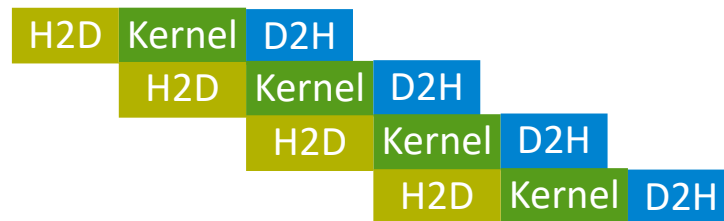


Concurrency Example

- Serial



- Concurrent – overlap kernel and data copy



H2D = Host to Device data copy

D2H = Device to Host data copy

Concurrency Example

- Serial



CPU Function

GPU Kernel

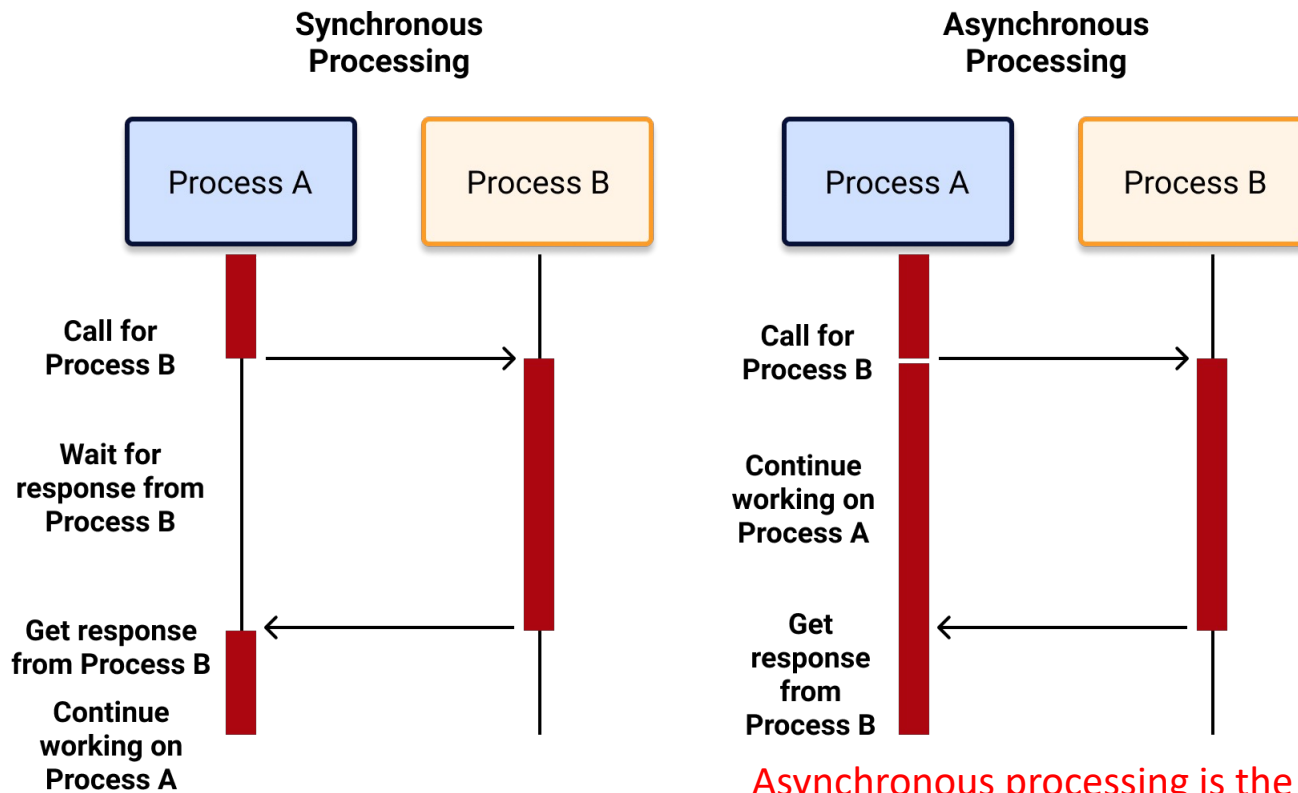
- Concurrent – overlap CPU and GPU kernels



CPU Function

GPU Kernel

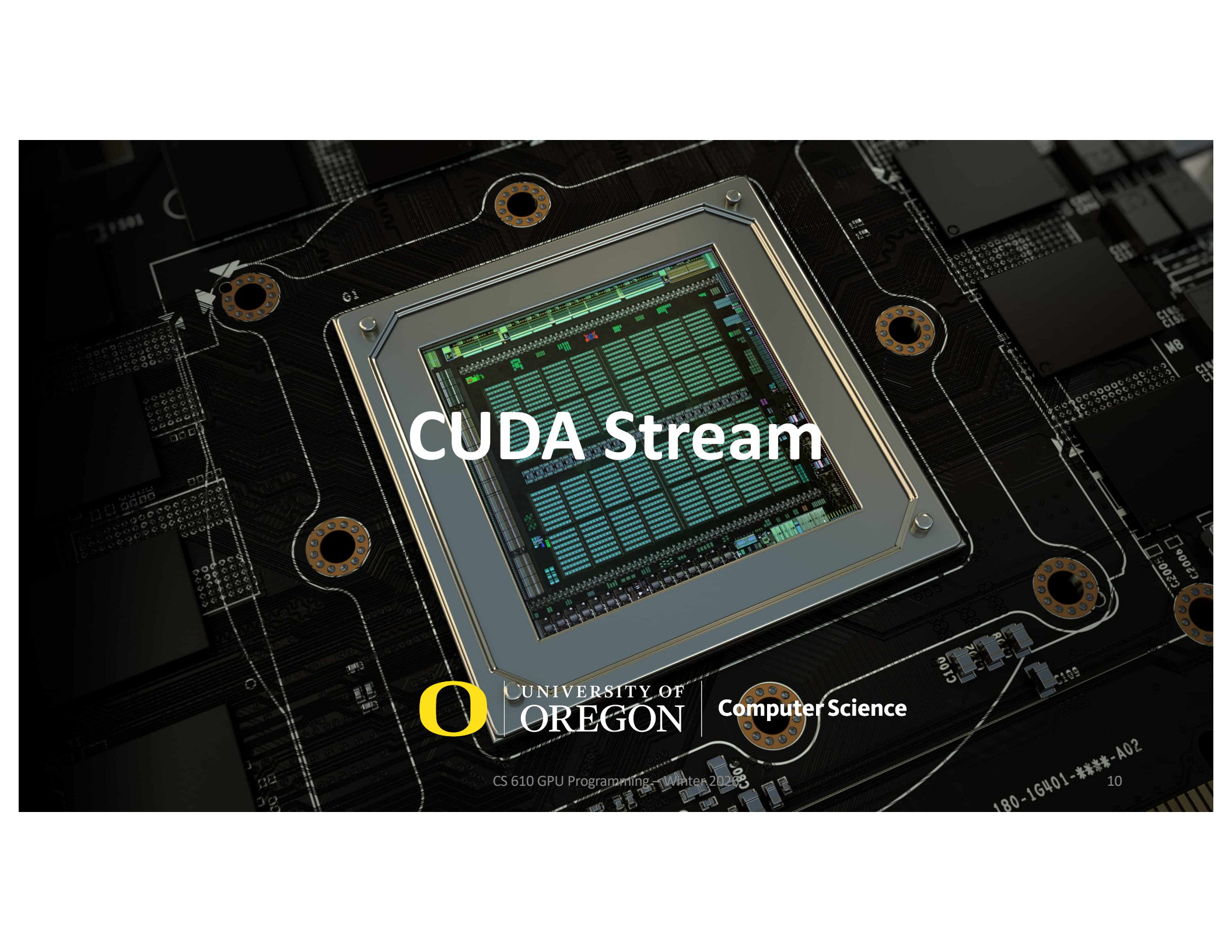
Synchronous vs. Asynchronous processing



Asynchronous processing is the key for achieving concurrent execution

Alternative terms

- Synchronous operations = **blocking** operations
- Asynchronous operations = **non-blocking** operations



CUDA Stream



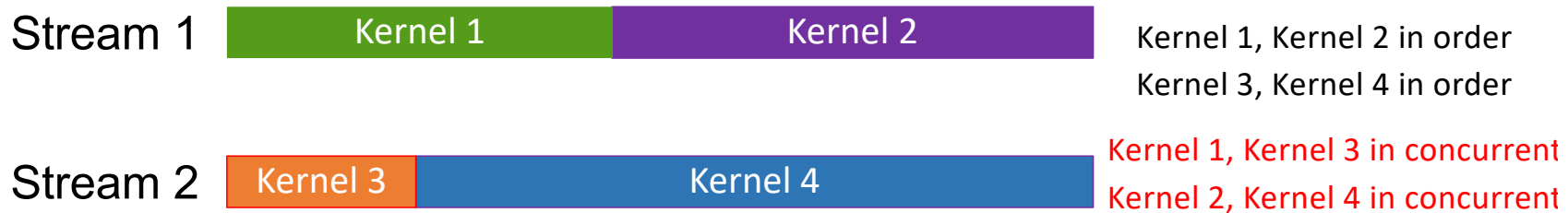
UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

CUDA Streams

- Stream = software executing queue
 - Operations in the same streams are guaranteed to be executed in the insertion order
 - Operations from different streams **may** run concurrently
 - **No guarantee** on the execution order for different streams



Create Streams

Declare and create streams before any CUDA operations

```
cudaStream_t stream  
cudaStreamCreate(&stream);
```

```
cudaStream_t stream[2];  
for (int i = 0; i < 2; ++i)  
    cudaStreamCreate(&stream[i]);
```

Destroy Streams

Destroy all streams before program exits.

```
cudaStreamDestroy(stream);
```

```
for (int i = 0; i < 2; ++i)  
    cudaStreamDestroy (stream[i]);
```



Asynchronous kernels with stream

4th-parameter



```
// Kernel invocation with N threads ,  
Kernel Name<<<number of blocks, thread per block, shared mem, stream>>>(...parameters...);
```

Examples

```
Kernel<<<16, 256, 0, stream>>>(...parameters...);  
CPU_Function(...parameters...);
```

Host thread:

Stream:

CPU Function

GPU Kernel

overlap CPU function and GPU kernel

Asynchronous kernels with stream

```
// Kernel invocation with N threads ,  
Kernel Name<<<number of blocks, thread per block, shared mem, stream>>>(...parameters...);
```

Examples

```
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
Kernel2<<<16, 256, 0, stream2>>>(...parameters...);
```

stream1:  Kernel 1
stream2:  Kernel 2

overlap two GPU kernels

Asynchronous memory copy with stream

```
// copy num bytes from src to dst on stream  
cudaMemcpyAsync(dst, src, num bytes, cudaMemcpyDefault, stream)
```

Examples

```
int * d_data = ...  
int * h_data = ...  
cudaMemcpyAsync(d_data, h_data, n * sizeof(int), cudaMemcpyDefault, stream)  
CPU_Function(...parameter...)
```

host thread:

CPU_Function

stream:

H2D

overlap two host function with memory copy

Asynchronous memory copy with stream

```
// copy num bytes from src to dst on stream  
cudaMemcpyAsync(dst, src, num bytes, cudaMemcpyDefault, stream)
```

Examples

```
int * d_data = ...  
int * h_data = ...  
cudaMemcpyAsync(d_data, h_data, n * sizeof(int), cudaMemcpyDefault, stream1)  
Kernel<<<16, 256, 0, stream2>>>(...parameters...);
```

stream1: H2D

stream2: Kernel

overlap GPU kernel with memory copy

Asynchronous memory copy with stream

```
// copy num bytes from src to dst on stream  
cudaMemcpyAsync(dst, src, num bytes, cudaMemcpyDefault, stream)
```

Examples

```
int * d_data1, * d_data2 = ...  
int * h_data1, * h_data2 = ...  
cudaMemcpyAsync(d_data1, h_data1, n * sizeof(int), cudaMemcpyDefault, stream1);  
cudaMemcpyAsync(h_data2, d_data2, n * sizeof(int), cudaMemcpyDefault, stream2)
```

stream1:  H2D

stream2:  D2H

overlap two memory copy operations

Asynchronous memory allocation with stream

```
// allocate num bytes on stream  
cudaMallocAsync(ptr, num bytes, stream)
```

Examples

```
int * d_data = ...  
int * h_data = ...  
cudaMallocAsync(d_data, num bytes, stream)  
cudaMemcpyAsync(d_data, h_data, n * sizeof(int), cudaMemcpyDefault, stream)  
CPU_Function(..parameters...)
```

host thread:

CPU_Function

stream:

malloc

H2D

Asynchronous memory free with stream

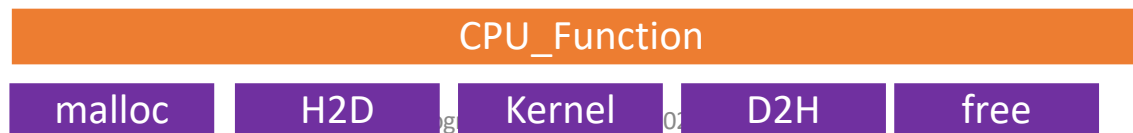
```
// free memory on stream  
cudaFreeAsync(ptr, stream)
```

Examples

```
int * d_data = ...  
int * h_data = ...  
cudaMallocAsync(d_data, num bytes, stream)  
cudaMemcpyAsync(d_data, h_data, n * sizeof(int), cudaMemcpyDefault, stream)  
Kernel<<<16, 256, 0, stream >>>(...parameters...);  
cudaMemcpyAsync(h_data, h_data, n * sizeof(int), cudaMemcpyDefault, stream)  
cudaFreeAsync(d_data)  
CPU_Function(..parameters...)
```

host thread:

stream:



Asynchronous memory set with stream

```
// free memory on stream  
cudaMemsetAsync(ptr, value, count, stream)
```

Examples

```
int * d_data = ...  
int * h_data = ...  
cudaMallocAsync(d_data, num bytes, stream)  
cudaMemsetAsync(d_data, 0, num bytes, stream)  
cudaMemcpyAsync(d_data, h_data, n * sizeof(int), cudaMemcpyDefault, stream)  
Kernel<<<16, 256, 0, stream >>>(...parameters...);  
cudaMemcpyAsync(h_data, h_data, n * sizeof(int), cudaMemcpyDefault, stream)  
cudaFreeAsync(d_data)  
CPU_Function(..parameters...)
```

host thread:

CPU_Function

stream:

malloc

memset

H2D

Kernel

D2H

free

Amount of Concurrency

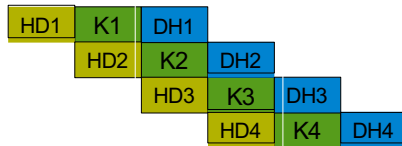
- Serial (1x)



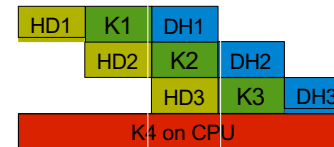
- 2-way concurrency (up to 2x)



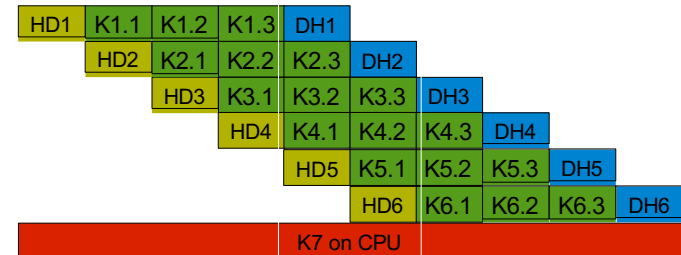
- 3-way concurrency (up to 3x)



- 4-way concurrency (3x+)



- 4+ way concurrency



Synchronize Streams

- Wait for all tasks in a stream

`cudaStreamSynchronize(stream)`

Examples

```
Kernel<<<16, 256, 0, stream>>>(...parameters...);  
CPU_Function1(...parameters...);  
cudaStreamSynchronize(stream);  
CPU_Function2(...parameters...);
```

Host thread:

CPU Function1

CPU Function2

Stream:

GPU Kernel

synchronize stream

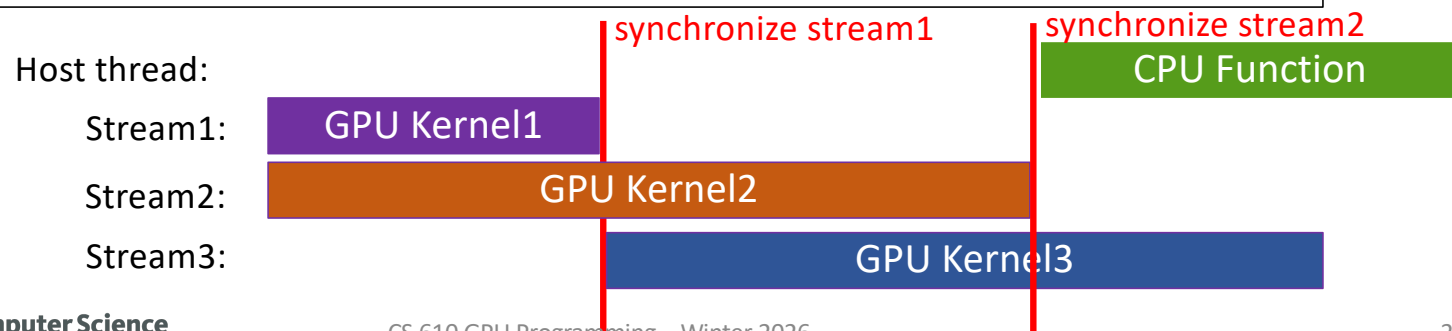
Synchronize Streams

- Wait for all tasks in a stream

`cudaStreamSynchronize(stream)`

Examples

```
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
Kernel2<<<16, 256, 0, stream2>>>(...parameters...);  
cudaStreamSynchronize(stream1);  
Kernel3<<<16, 256, 0, stream3>>>(...parameters...);  
cudaStreamSynchronize(stream2);  
CPU_Function(...parameters...);
```



Synchronize Device

- Wait for all tasks on a device (all streams)

`cudaDeviceSynchronize()`

Examples

```
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
Kernel2<<<16, 256, 0, stream2>>>(...parameters...);  
Kernel3<<<16, 256, 0, stream3>>>(...parameters...);  
cudaDeviceSynchronize();  
CPU_Function(...parameters...);
```

Host thread:

Stream1:

GPU Kernel1

Stream2:

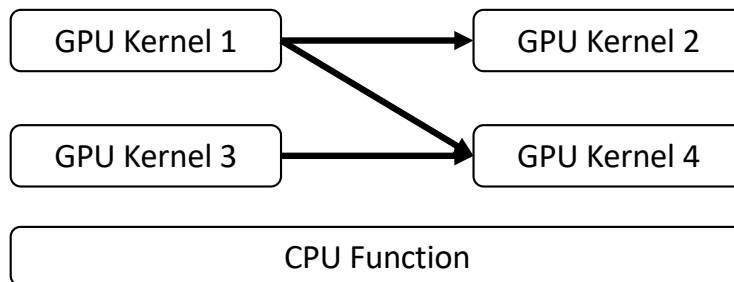
GPU Kernel2

Stream3:

GPU Kernel3

synchronize device

CPU Function





CUDA Events

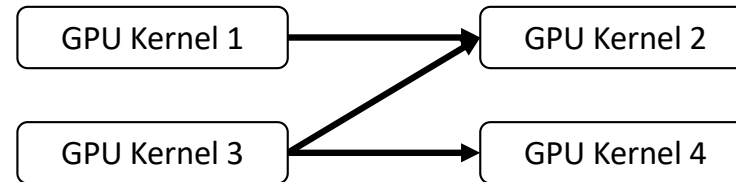


UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

CUDA Events



How to enforce the dependency without causing delay?

CUDA Events

Does this work?

```
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
Kernel3<<<16, 256, 0, stream2>>>(...parameters...);  
cudaStreamSynchronize(stream2);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);  
Kernel4<<<16, 256, 0, stream2>>>(...parameters...);
```

Stream1:

GPU Kernel1

GPU Kernel2

Stream2:

GPU Kernel3

GPU Kernel4

Kernel 4 is delayed!

CUDA Events

Does this work?

```
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
Kernel3<<<16, 256, 0, stream2>>>(...parameters...);  
Kernel4<<<16, 256, 0, stream2>>>(...parameters...);  
cudaStreamSynchronize(stream2);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);
```

Stream1:

GPU Kernel1

GPU Kernel2

Stream2:

GPU Kernel3

GPU Kernel4

Kernel 2 is delayed!

CUDA Events

- Adding an event to a stream
- Achieving finer-grain synchronization

```
cudaEvent_t my_event;  
cudaEventCreate(& my_event);  
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
Kernel3<<<16, 256, 0, stream2>>>(...parameters...);  
cudaEventRecord(my_event, stream2);  
Kernel4<<<16, 256, 0, stream2>>>(...parameters...);  
cudaEventSynchronize(my_event); // only waits Kernel3  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);
```



Create CUDA Events

- `cudaEvent_t event;`
- `cudaEventCreate(&event);`

Examples

```
cudaEvent_t event1, event2;  
cudaEventCreate(&event1);  
cudaEventCreate(&event2);
```

Destroy CUDA Events

```
cudaEventDestroy(event);
```

Examples

```
cudaEventDestroy(event1);  
cudaEventDestroy(event2);
```

Record CUDA Events

```
cudaEventRecord(event, stream);
```

Examples

```
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventRecord(event1, stream1);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);
```

Add an event in between kernel1 and kernel2



Synchronize CUDA Events

```
cudaEventSynchronize(event);
```

Wait for all operations **preceding** the event

Examples

```
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventRecord(event1, stream1);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventSynchronize(event1)
```

Add an event in between kernel1 and kernel2
Wait for kernel1 only



Query CUDA Events

`cudaEventQuery(event);`

Check the status of event without waiting

return `cudaSuccess`: all tasks preceding event have completed

return `cudaErrorNotReady`: all tasks preceding event have not completed

Examples

```
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventRecord(event1, stream1);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventQuery(event1)
```

`cudaErrorNotReady` `cudaSuccess` `cudaSuccess`

Timing using CUDA Events

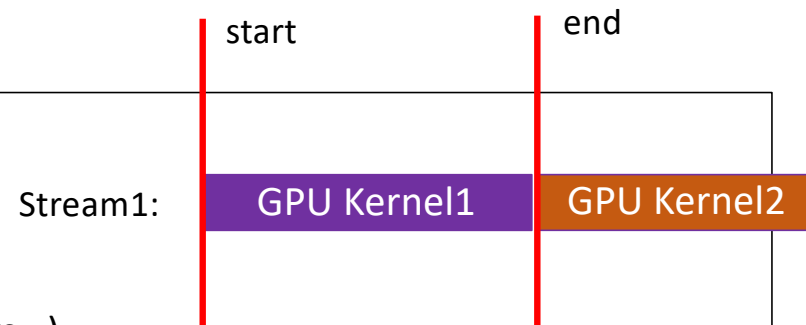
`cudaEventElapsedTime ( float* ms, event_start, event_end)`

Timing operations without blocking other operations

Must synchronize `event_end` before calculating the time

Examples

```
cudaEvent_t start, end;
cudaEventCreate(&start);
cudaEventCreate(&end);
cudaEventRecord(start, stream1);
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);
cudaEventRecord(end, stream1);
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);
cudaEventSynchronize(end);
float elapsedTime;
cudaEventElapsedTime(&elapsedTime, start, stop);
std::cout << elapsedTime/1000.0 << "seconds\n"; // ms to s
```



Synchronize CUDA Events with stream

- Previously, we learnt that `cudaEventSynchronize(event)` blocks the host thread for the completion of the event.

Examples

```
cudaEvent_t my_event;  
cudaEventCreate(& my_event);  
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventRecord(my_event, stream1);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventSynchronize(my_event);  
CPU_Function(...parameters...);
```

sync my_event without delaying kernel2

Host thread:

Stream1:

GPU Kernel1

GPU Kernel2

CPU Function

Synchronize CUDA Events with stream

- Previously, we learnt that `cudaEventSynchronize(event)` blocks the host thread for the completion of the event.

Examples

```
cudaEvent_t my_event;  
cudaEventCreate(& my_event);  
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventRecord(my_event, stream1);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventSynchronize(my_event);  
Kernel3<<<16, 256, 0, stream2>>>(...parameters...);
```

sync my_event without delaying kernel2

Host thread:

Stream1:

GPU Kernel1

GPU Kernel2

Stream2:

GPU Kernel3



Synchronize CUDA Events with stream

- Previously, we learnt that `cudaEventSynchronize(event)` blocks the host thread for the completion of the event.

What if we have expensive CPU function?

Examples

```
cudaEvent_t my_event;  
cudaEventCreate(& my_event);  
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventRecord(my_event, stream1);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventSynchronize(my_event);  
Kernel3<<<16, 256, 0, stream2>>>(...parameters...);
```

sync my_event without delaying kernel2

Host thread:

Stream1:

GPU Kernel1

GPU Kernel2

Stream2:

GPU Kernel3



Synchronize CUDA Events with stream

- Previously, we learnt that `cudaEventSynchronize(event)` blocks the host thread for the completion of the event.

What if we have expensive CPU function?

Examples

```
CPU_Function(...parameters...)
cudaEvent_t my_event;
cudaEventCreate(& my_event);
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);
cudaEventRecord(my_event, stream1);
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);
cudaEventSynchronize(my_event);
Kernel3<<<16, 256, 0, stream2>>>(...parameters...);
```

Host thread:

CPU Function

Stream1:

GPU Kernel1

GPU Kernel2

Delay all GPU kernels!

Synchronize CUDA Events with stream

- Previously, we learnt that `cudaEventSynchronize(event)` blocks the host thread for the completion of the event.

What if we have expensive CPU function?

Examples

```
cudaEvent_t my_event;  
cudaEventCreate(& my_event);  
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
CPU_Function(...parameters...)  
cudaEventRecord(my_event, stream1);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventSynchronize(my_event);  
Kernel3<<<16, 256, 0, stream2>>>(...parameters...);
```



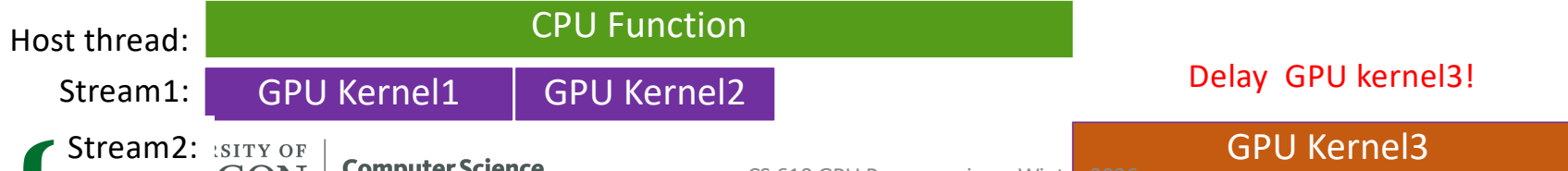
Synchronize CUDA Events with stream

- Previously, we learnt that `cudaEventSynchronize(event)` blocks the host thread for the completion of the event.

What if we have expensive CPU function?

Examples

```
cudaEvent_t my_event;  
cudaEventCreate(& my_event);  
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventRecord(my_event, stream1);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);  
CPU_Function(...parameters...)  
cudaEventSynchronize(my_event);  
Kernel3<<<16, 256, 0, stream2>>>(...parameters...);
```



Synchronize CUDA Events with stream

- Previously, we learnt that `cudaEventSynchronize(event)` blocks the host thread for the completion of the event.

What if we have expensive CPU function?

Examples

```
cudaEvent_t my_event;  
cudaEventCreate(& my_event);  
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventRecord(my_event, stream1);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventSynchronize(my_event);  
Kernel3<<<16, 256, 0, stream2>>>(...parameters...);  
CPU_Function(...parameters...)
```

The problem is
`cudaEventSynchronize` blocks
the CPU thread, we cannot
do anything else on CPU

Host thread:

CPU Function

Stream1:

GPU Kernel1

GPU Kernel2

Stream2:

GPU Kernel3

Delay CPU function!

Synchronize CUDA Events with stream

```
cudaStreamWaitEvent(stream2, my_event);
```

- Makes all the commands added later to the given stream delay their execution until the given event has complete. The event can be on a different stream
- This function **does not block** the CPU threads

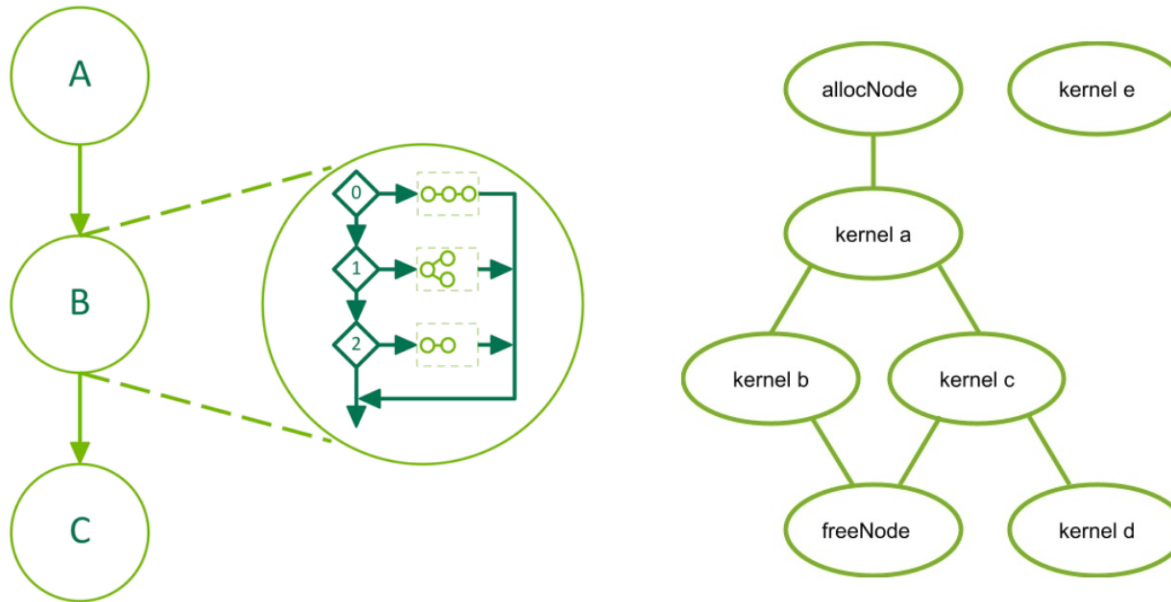
Examples

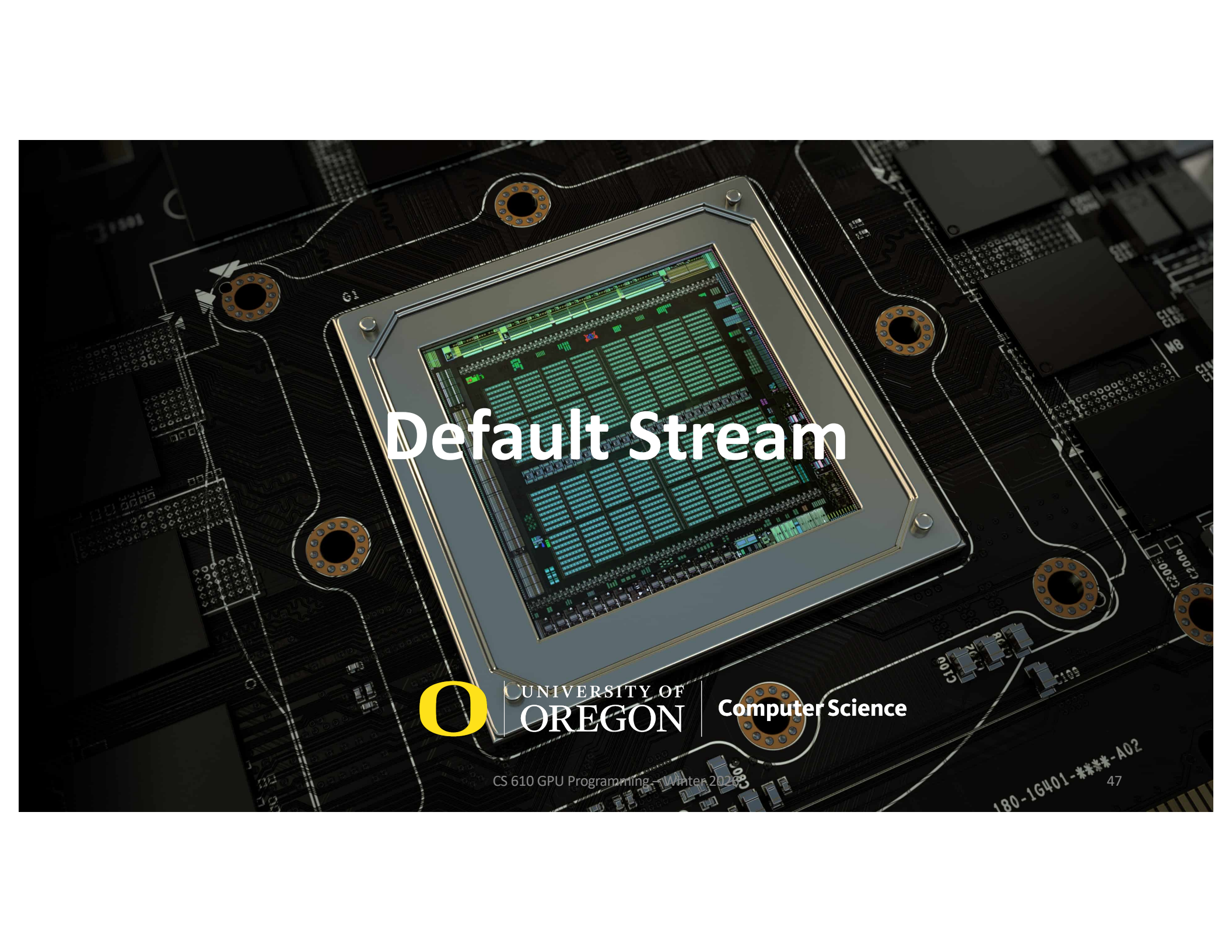
```
cudaEvent_t my_event;  
cudaEventCreate(& my_event);  
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
cudaEventRecord(my_event, stream1);  
Kernel2<<<16, 256, 0, stream1>>>(...parameters...);  
cudaStreamWaitEvent(stream2, my_event);  
Kernel3<<<16, 256, 0, stream2>>>(...parameters...);  
CPU_Function(...parameters...)
```



More complex dependencies?

- CUDA Graphs API: <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html>





Default Stream



UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

Default Stream (aka Null Stream)

- Default stream is **implicitly** used when no stream is specified
- Kernel call is always **asynchronous (non-blocking)** relative to host
- Kernel execution will be **concurrent** relative to host thread

Example

```
Kernel<<<16, 256>>>(...parameters...);  
CPU_Function(...parameters...);
```

Host thread:

CPU Function

Default Stream:

GPU Kernel

overlap CPU function and GPU kernel

Default Stream (aka Null Stream)

- Default stream is **implicitly** used when no stream is specified

Example

```
Kernel<<<16, 256>>>(...parameters...);  
cudaMalloc(...parameters...);  
cudaMemcpy(...parameters...);  
cudaFree(...parameters...);  
cudaMemset(...parameters...);
```

Default Stream (aka Null Stream)

- Default stream is **implicitly** used when no stream is specified
- Kernel call is always **asynchronous (non-blocking)** relative to host
- Kernel execution will be **concurrent** relative to host thread
- Multiple kernels are serialized

Example

```
Kernel1<<<16, 256>>>(...parameters...);  
Kernel2<<<16, 256>>>(...parameters...);  
CPU_Function(...parameters...);
```

Host thread:

CPU Function

Default Stream:

GPU Kernel1

GPU Kernel2

Default Stream (aka Null Stream)

- Default stream is **implicitly** used when no stream is specified
- Memory operations call is always **synchronous (blocking)** relative to **host and device**
- Memory operations will be serialized with both **CPU functions and GPU kernels**

Example

```
cudaMemcpy(...parameters...)  
CPU_Function(...parameters...);
```

Host thread:

Default Stream:

memcpy

CPU Function

Default Stream (aka Null Stream)

- Default stream is **implicitly** used when no stream is specified
- Memory operations call is always **synchronous (blocking)** relative to **host and device**
- Memory operations will be serialized with both **CPU functions and GPU kernels**

Example

```
CPU_Function1(...parameters...);  
cudaMemcpy(...parameters...)  
CPU_Function2(...parameters...);
```

Host thread:

CPU Function1

CPU Function2

Default Stream:

memcpy

Default Stream (aka Null Stream)

- Default stream is **implicitly** used when no stream is specified
- Memory operations call is always **synchronous (blocking)** relative to **host and device**
- Memory operations will be serialized with both **CPU functions and GPU kernels**

Example

```
Kernel1<<<16, 256>>>(...parameters...);  
cudaMemcpy(...parameters...)  
CPU_Function1(...parameters...);
```

Host thread:

Default Stream:

Computer Science

GPU Kernel 1

memcpy

CPU Function1

Default Stream (aka Null Stream)

- Default stream is **implicitly** used when no stream is specified
- Memory operations call is always **synchronous (blocking)** relative to **host and device**
- Memory operations will be serialized with both **CPU functions and GPU kernels**
- **As if `cudaDeviceSynchronize()` inserted before and after**

Example

```
Kernel1<<<16, 256>>>(...parameters...);  
CPU_Function1(...parameters...);  
cudaMemcpy(...parameters...)
```

Host thread:

CPU Function1

Default Stream:

GPU Kernel 1

memcpy

Default Stream (aka Null Stream)

- Default stream is **implicitly** used when no stream is specified
 - Memory operations call is always **synchronous (blocking)** relative to **host and device**
 - Memory operations will be serialized with both **CPU functions and GPU kernels**
 - **As if `cudaDeviceSynchronize()` inserted before and after**
- Example

```
Kernel1<<<16, 256>>>(...parameters...);  
CPU_Function1(...parameters...);  
cudaMemcpy(...parameters...)  
Kernel2<<<16, 256>>>(...parameters...);
```

Host thread:

CPU Function1

Default Stream:

GPU Kernel 1

memcpy

GPU Kernel 2

Default Stream (aka Null Stream)

- Default stream is **implicitly** used when no stream is specified
- Memory operations call is always **synchronous (blocking)** relative to **host and device**
- Memory operations will be serialized with both **CPU functions and GPU kernels**
- **As if `cudaDeviceSynchronize()` inserted before and after**

Example

```
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
CPU_Function1(...parameters...);  
cudaMemcpy(...parameters...)  
Kernel2<<<16, 256>>>(...parameters...);
```

Host thread:

CPU Function1

Stream1:

GPU Kernel 1

Default Stream:

memcpy

GPU Kernel 2



UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming – Winter 2026

Default Stream (aka Null Stream)

- Default stream is **implicitly** used when no stream is specified
- Memory operations call is always **synchronous (blocking)** relative to **host and device**
- Memory operations will be serialized with both **CPU functions and GPU kernels**
- **As if `cudaDeviceSynchronize()` inserted before and after**

Example

```
Kernel1<<<16, 256, 0, stream1>>>(...parameters...);  
CPU_Function1(...parameters...);  
cudaMalloc(...parameters...)  
Kernel2<<<16, 256>>>(...parameters...);
```

Host thread:

CPU Function1

Stream1:

GPU Kernel 1

Default Stream:

malloc

GPU Kernel 2



UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming – Winter 2026

Requirement for Achieving Async. Memory Copy



Asynchronous memory copy with stream

```
// copy num bytes from src to dst on stream  
cudaMemcpyAsync(dst, src, num bytes, cudaMemcpyDefault, stream)
```

Special memory allocation is
necessary for achieving
asynchronous memory copy

Examples

```
int * d_data = ...  
int * h_data = ...  
cudamemcpyAsync(d_data, h_data, n * sizeof(int), cudaMemcpyDefault, stream1)  
Kernel<<<16, 256, 0, stream2>>>(...parameters...);
```

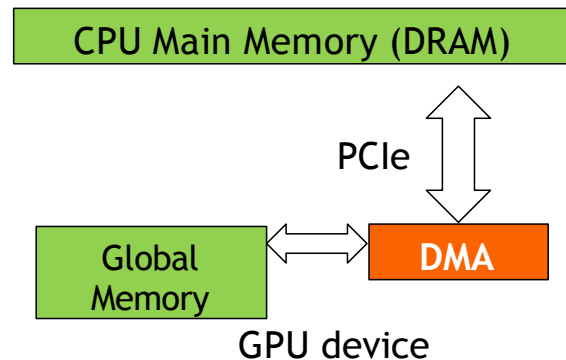
stream1: H2D

stream2: Kernel

overlap GPU kernel with memory copy

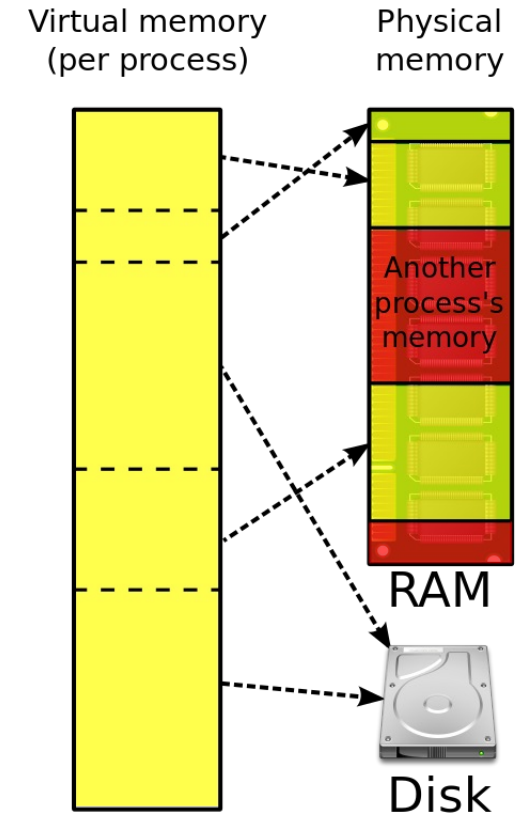
CPU-GPU Data Transfer using DMA

- DMA (Direct Memory Access) hardware
 - Used by `cudaMemcpy()` for better efficiency
 - Used by `cudaMemcpyAsync()` for achieving **asynchronous** execution
 - Frees CPU for other tasks (achieving **asynchronous**)
 - Hardware unit specialized to transfer a number of bytes requested by OS
 - Uses system interconnect, typically PCIe or NVLINK in today's systems



Virtual Memory Management

- Modern CPUs use virtual memory management
 - Many virtual memory spaces mapped into a single physical memory
 - Virtual addresses (pointer values) are translated into physical addresses
- Variables and data structures are **not always** in the physical memory
 - Each virtual address space is divided into pages that are mapped into and out of the physical memory
 - Virtual memory pages can be mapped out of the physical memory (page-out) to make room
 - Whether or not a variable is in the physical memory is checked at address translation time



Data Transfer and Virtual Memory

- DMA uses physical addresses
 - When `cudaMemcpy()` copies an array, it is implemented as one or more DMA transfers
 - Address is translated and page presence checked for the entire source and destination regions at the beginning of each DMA transfer
 - No address translation for the rest of the same DMA transfer so that high efficiency can be achieved
- The OS could accidentally page-out the data that is being read or written by a DMA and page-in another virtual page into the same physical location

Pinned Memory and DMA Data Transfer

- Pinned memory are virtual memory pages that are specially marked so that they cannot be paged out
- Allocated with a special system API function call
- a.k.a. Page Locked Memory, Locked Pages, etc.
- CPU memory that serve as the source or destination of a DMA transfer must be allocated as pinned memory

CUDA data transfer uses pinned memory.

- The DMA used by `cudaMemcpy()` requires that any source or destination in the host memory is allocated as pinned memory
- If a source or destination of a `cudaMemcpy()` in the host memory is not allocated in pinned memory, it needs to be first **copied** to a pinned memory – **extra overhead**
- `cudaMemcpy()` is faster if the host memory source or destination is **allocated** in pinned memory since **no extra copy** is needed
- Host memory must be already pinned to achieve asynchronous memory copy

Allocate/Free Pinned Memory

- `cudaHostAlloc()`, three parameters
 - Address of pointer to the allocated memory
 - Size of the allocated memory in bytes
 - Option – use `cudaHostAllocDefault` for now
- `cudaFreeHost()`, one parameter
 - Pointer to the memory to be freed

Using Pinned Memory in CUDA

- Use the allocated pinned memory and its pointer the same way as those returned by `malloc()`;
- The only difference is that the allocated memory cannot be paged by the OS
- The `cudaMemcpy()` function should be about 2X faster with pinned memory
- Pinned memory is a limited resource
 - over-subscription can have serious consequences

Example

```
int main()
{
    float *h_A, d_A, *h_B, *d_B;

    cudaHostAlloc((void **) &h_A, N* sizeof(float), cudaHostAllocDefault);

    h_B = (float*)malloc(N* sizeof(float));
    cudaMalloc((void**)d_A, N* sizeof(float));
    cudaMalloc((void**)d_B, N* sizeof(float));

    cudaMemcpy(d_B, h_B, n * sizeof(int), cudaMemcpyDefault); // slow, synchronous copy
    cudaMemcpy(d_A, h_A, n * sizeof(int), cudaMemcpyDefault); // 2X faster, synchronous copy

    cudaMemcpyAsync(d_B, h_B, n * sizeof(int), cudaMemcpyDefault, stream) // slow, synchronous copy
    cudaMemcpyAsync(d_A, h_A, n * sizeof(int), cudaMemcpyDefault, stream) // 2X faster, asynchronous copy
}
```

Explicit Synchronization

- **Synchronize everything**
 - `cudaDeviceSynchronize ()`
 - Blocks host until all issued CUDA calls are complete
- **Synchronize w.r.t. a specific stream**
 - `cudaStreamSynchronize ( streamid )`
 - Blocks host until all CUDA calls in streamid are complete
- **Synchronize using Events**
 - Create specific 'Events', within streams, to use for synchronization
 - `cudaEventRecord ( event, streamid )`
 - `cudaEventSynchronize ( event )`
 - `cudaStreamWaitEvent ( stream, event )`
 - `cudaEventQuery ( event )`



Explicit Synchronization Example

- **Resolve using an event**

```
{
    cudaEvent_t event;
    cudaEventCreate (&event);                // create event

    cudaMemcpyAsync ( d_in, in, size, H2D, stream1 );    // 1) H2D copy of new input
    cudaEventRecord (event, stream1);                // record event

    cudaMemcpyAsync ( out, d_out, size, D2H, stream2 );    // 2) D2H copy of previous result

    cudaStreamWaitEvent ( stream2, event );            // wait for event in stream1
    kernel <<< , , , stream2 >>> ( d_in, d_out );        // 3) must wait for 1 and 2

    asynchronousCPUmethod ( ... )                // Async GPU method
}
```



Implicit Synchronization

- **These operations implicitly synchronize all other CUDA operations**
 - **Page-locked memory allocation**
`cudaMallocHost`
`cudaHostAlloc`
 - **Device memory allocation**
`cudaMalloc`
 - **Non-Async version of memory operations**
`cudaMemcpy` (no Async suffix)
`cudaMemset` (no Async suffix)
 - **GPU configuration API**
`cudaDeviceSetCacheConfig`

Concurrency

- **The ability to perform multiple CUDA operations simultaneously**
(beyond multi-threaded parallelism)
 - **CUDA Kernel <<<>>>**
 - **cudaMemcpyAsync (HostToDevice)**
 - **cudaMemcpyAsync (DeviceToHost)**
 - **Operations on the CPU**

Example – Batched matrix multiplication

- **CPU** (4core Westmere x5670 @2.93 GHz, MKL)

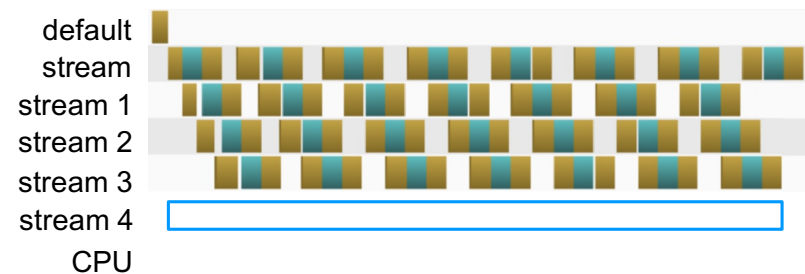
- **43 Gflops**

DGEMM: $m=n=8192$, $k=288$

- **GPU** (C2070)

- **Serial : 125 Gflops (2.9x)**
- **2 way : 177 Gflops (4.1x)**
- **3 way : 262 Gflops (6.1x)**

Nvidia Visual Profiler (nvvp)



- **Obtain maximum performance by leveraging concurrency**
- **All communication hidden – effectively removes device memory size limitations**

Requirements for Concurrency

- **CUDA operations must be in different, non-default, streams**
- **cudaMemcpyAsync with host from 'pinned' memory**
 - Page-locked memory
 - Allocated using `cudaMallocHost()` or `cudaHostAlloc()`
- **Sufficient resources must be available**
 - `cudaMemcpyAsync`s in different directions Device
 - resources (SMEM, registers, blocks, etc.)

Which operations can be done concurrently?

```
cudaMalloc ( &dev1, size ) ;  
double* host1 = (double*) malloc ( &host1, size ) ;  
...
```

```
cudaMemcpy ( dev1, host1, size, H2D ) ;  
kernel2 <<< grid, block, 0 >>> ( ..., dev2, ... ) ;  
kernel3 <<< grid, block, 0 >>> ( ..., dev3, ... ) ;  
cudaMemcpy ( host4, dev4, size, D2H ) ;  
...
```



**completely
synchronous**

- **All CUDA operations in the default stream are synchronous**

Which operations can be done concurrently?

```
cudaMalloc ( &dev1, size ) ;  
double* host1 = (double*) malloc ( &host1, size ) ;  
...
```

```
cudaMemcpy ( dev1, host1, size, H2D ) ;  
kernel2 <<< grid, block >>> ( ..., dev2, ... ) ;  
some_CPU_method () ;  
kernel3 <<< grid, block >>> ( ..., dev3, ... ) ;  
cudaMemcpy ( host4, dev4, size, D2H ) ;  
...
```



**potentially
overlapped**

- **GPU kernels are asynchronous with host by default**



Which operations can be done concurrently?

```
cudaStream_t stream1, stream2, stream3, stream4 ;
cudaStreamCreate ( &stream1 );
...
cudaMalloc ( &dev1, size ) ;
cudaMallocHost ( &host1, size ) ;
...
cudaMemcpyAsync ( dev1, host1, size, H2D, stream1 ) ;
kernel2 <<< grid, block, 0, stream2 >>> ( ..., dev2, ... ) ;
kernel3 <<< grid, block, 0, stream3 >>> ( ..., dev3, ... ) ;
cudaMemcpyAsync ( host4, dev4, size, D2H, stream4 ) ;
some_CPU_method ();
...
```

// pinned memory required on host

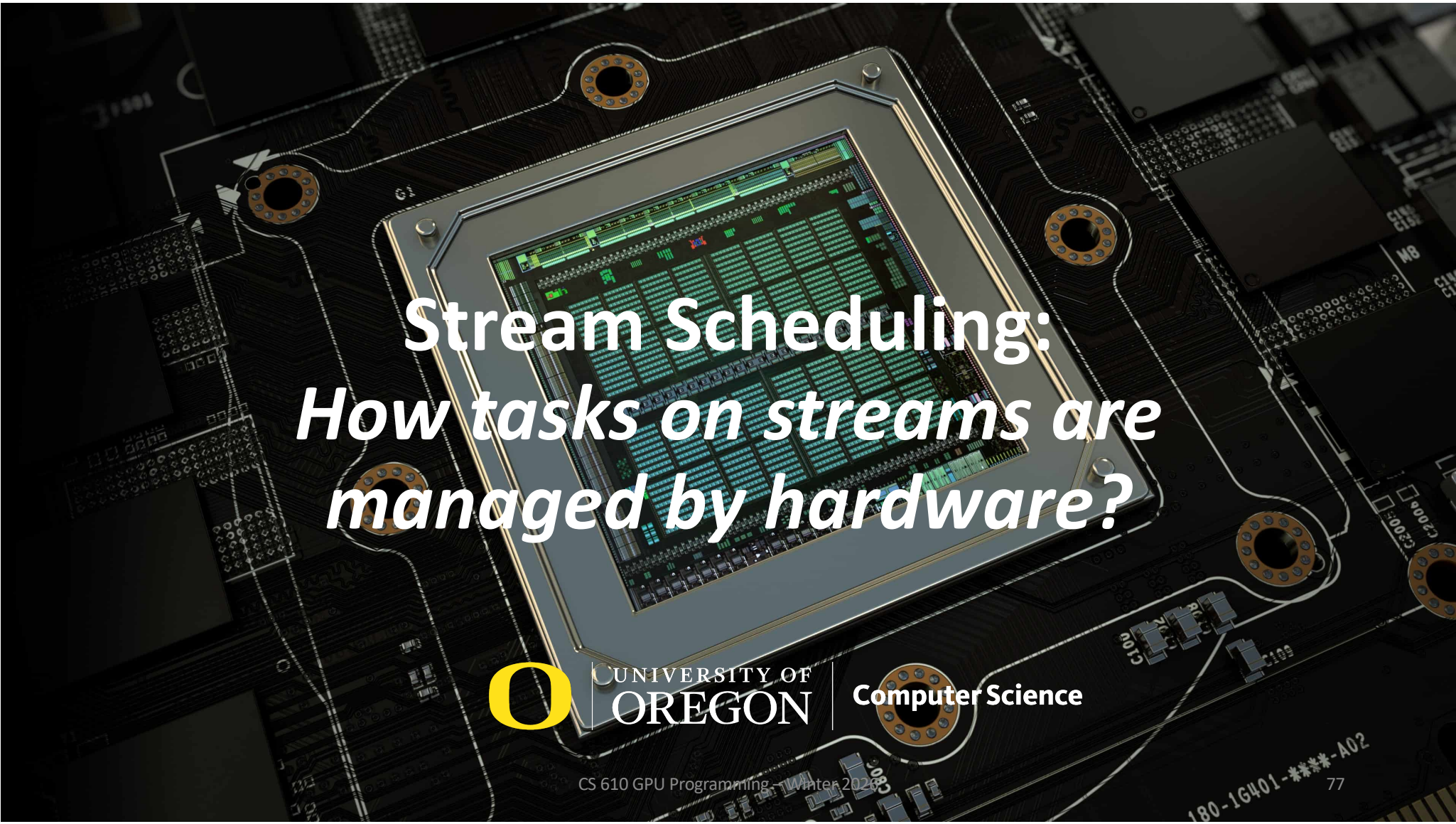


**potentially
overlapped**

Fully asynchronous / concurrent

Data used by concurrent operations should be independent





Stream Scheduling: *How tasks on streams are managed by hardware?*



UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

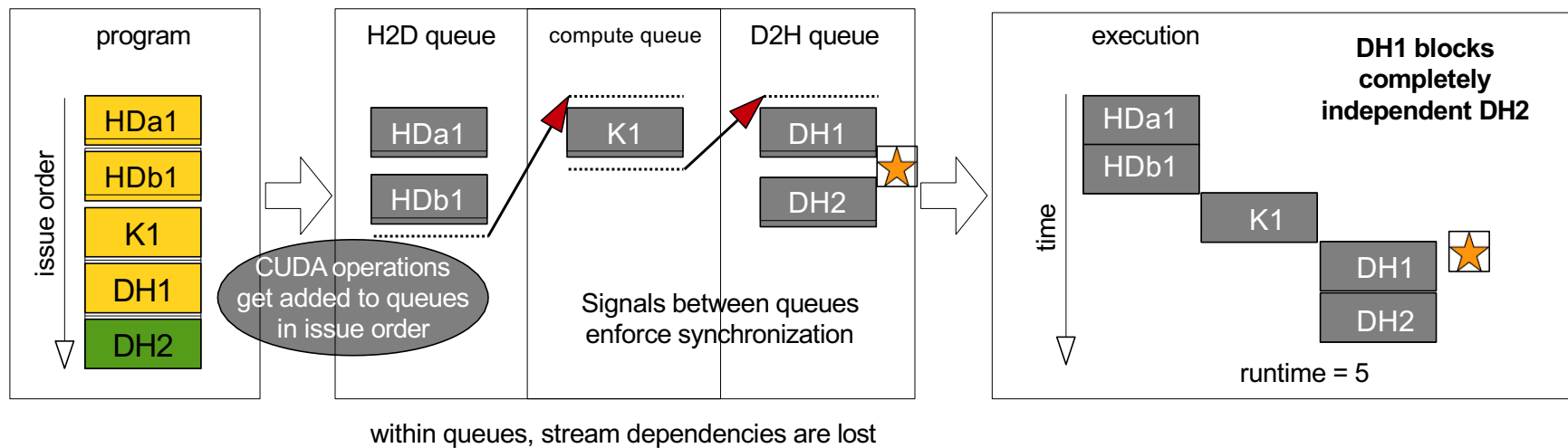
Stream Scheduling

- GPU hardware has 4 queues (could be more for newer GPUs)
 - 1 Compute Engine queue
 - 2 Copy Engine queues – one for H2D, one for D2H, one for D2D (A100)
- CUDA operations are dispatched to HW in the sequence they were issued
 - Placed in the relevant queue
 - Stream dependencies between engine queues are maintained, but lost within an engine queue
- A CUDA operation is dispatched from the engine queue if:
 - Preceding calls in the same stream have completed,
 - Preceding calls in the same queue have been dispatched, and
 - Resources are available
- CUDA kernels may be executed concurrently if they are in different streams
 - Threadblocks for a given kernel are scheduled if all threadblocks for preceding kernels have been scheduled and there still are SM resources available
- Note a blocked operation blocks all other operations in the queue, even in other streams



Example – Blocked Queue

- Two streams, stream 1 is issued first
 - Stream 1 : HDa1, HDb1, K1, DH1 (issued first)
 - Stream 2 : DH2 (completely independent of stream 1)

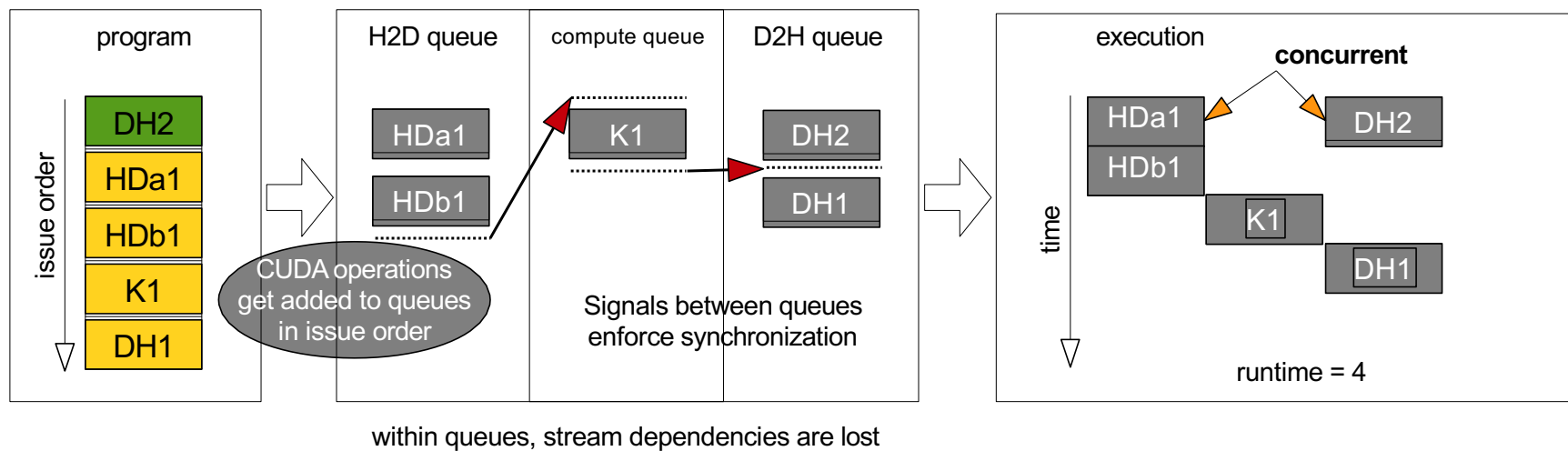


Example – Blocked Queue

- Two streams, stream 2 is issued first

- Stream 1 : HDa1, HDb1, K1, DH1
- Stream 2 : DH2 (issued first)

issue order matters!

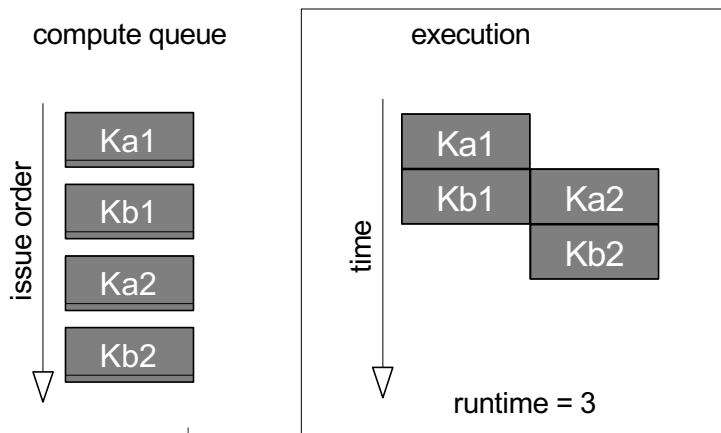


Example - Blocked Kernel

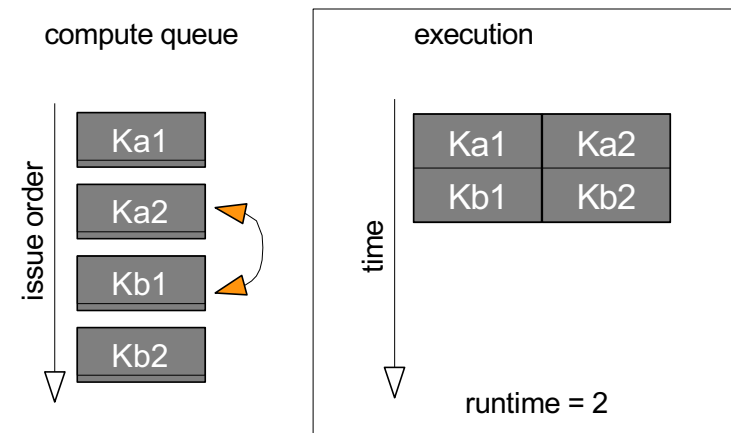
- Two streams – just issuing CUDA kernels
 - Stream 1 : Ka1, Kb1
 - Stream 2 : Ka2, Kb2
 - Kernels are similar size, fill $\frac{1}{2}$ of the SM resources

issue order matters!

- Issue **depth** first



- Issue **breadth** first



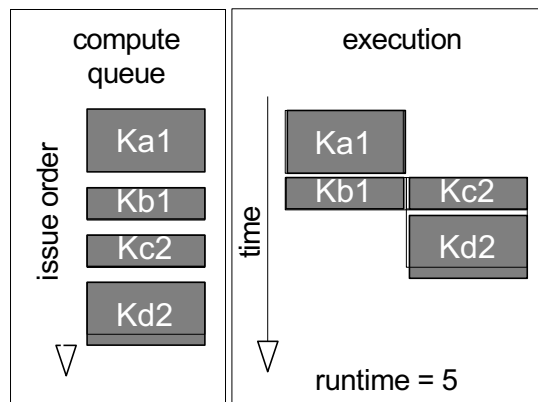
Example - Optimal concurrency can depend on kernel execution time

- Two streams – just issuing CUDA kernels – but kernels are different 'sizes'

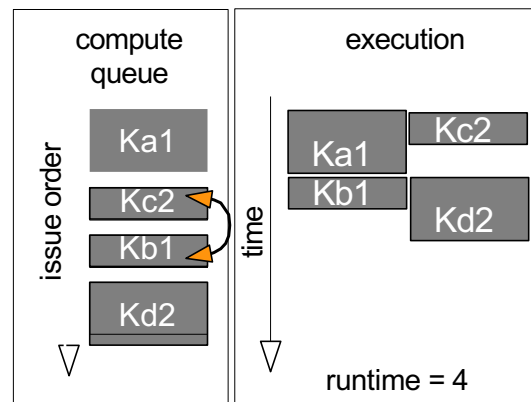
- Stream 1 : Ka1 {2}, Kb1 {1}
- Stream 2 : Kc2 {1}, Kd2 {2}
- Kernels fill $\frac{1}{2}$ of the SM resources

issue order matters!
execution time matters too!

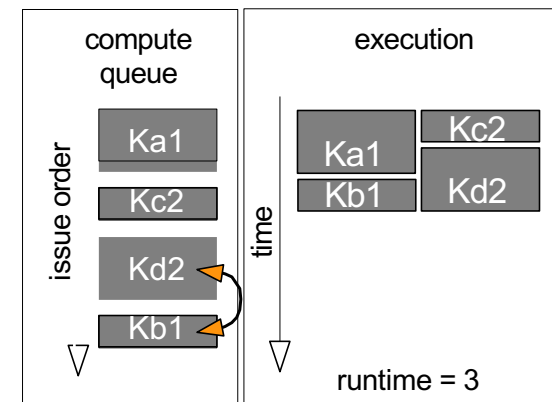
- Depth first

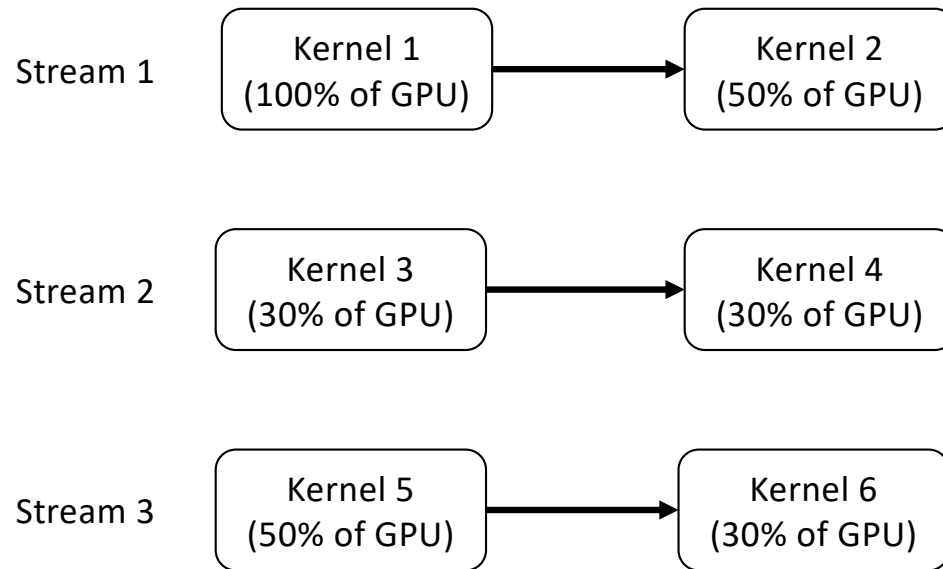


- Breadth first



- Custom



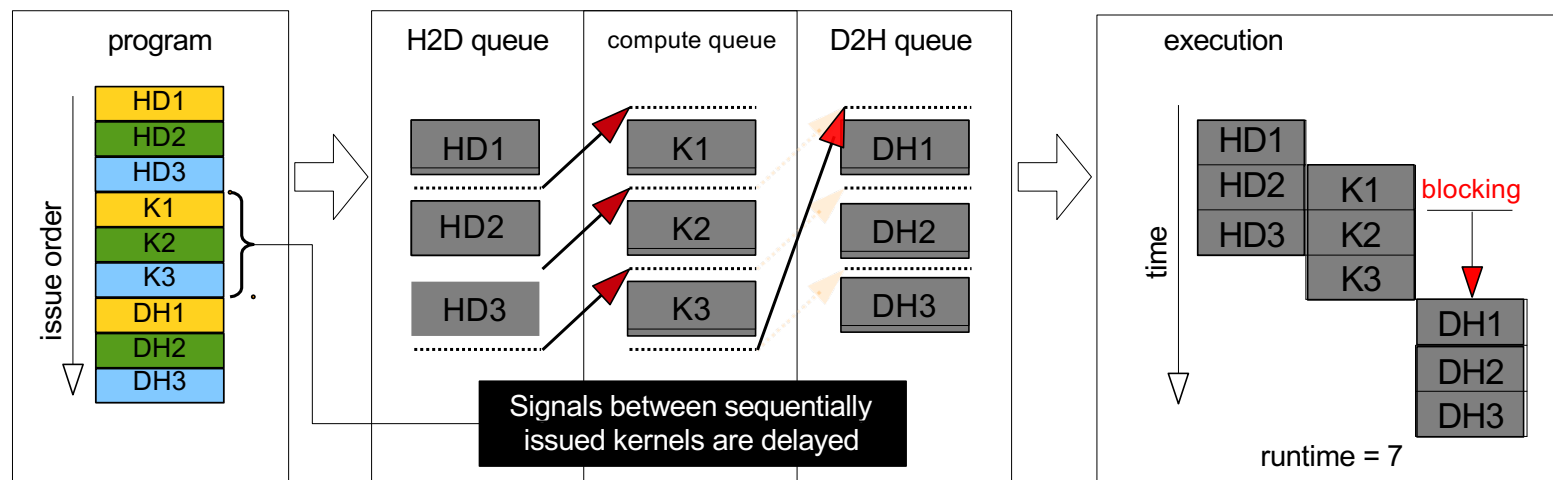


Concurrent Kernel Scheduling

- **Concurrent kernel scheduling is special**
- **Normally, a signal is inserted into the queues, after the operation, to launch the next operation in the same stream**
- **For the compute engine queue, to enable concurrent kernels, when compute kernels are issued sequentially, this signal is delayed until after the last sequential compute kernel**
- **In some situations this delay of signals can block other queues**

Example – Concurrent Kernels and Blocking

- Three streams, each performing (HD, K, DH)
- Breadth first
 - Sequentially issued kernels delay signals and block cudaMemcpy(D2H)



CUDA prioritizes compute progress over copy

Example – Concurrent Kernels and Blocking

- Three streams, each performing (HD, K, DH)
- Depth first: void computation prioritization

